

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	R KALAI AMUDHAN	Reg. No.:	192511292
Date:		Duration:	60 minutes

Code of Conduct

I R Kalai amudhan (192511292) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R KALAI AMUDHAN

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. PROBLEM ANALYSIS

The design emphasizes early identification of abnormal conditions so that farmers can take timely action rather than depending only on manual inspection.

The proposed system brings livestock health, production, feeding, and farm-level information into one centralized platform.

1.1 Problem Statement

The prototype is designed as a software foundation. Actual clinical diagnosis, production-grade AI prediction, and real-time sensor integration require validated datasets, domain expertise, and further testing.

A centralized platform can reduce fragmented record keeping by connecting cattle information, health observations, feeding data, milk production, and analytics in a common application.

The system addresses the difficulty of continuously observing every animal in a large dairy farm.

Sensor-based measurements can provide frequent observations that complement the farmer's normal inspection process.

The Intelligent AI-Based Smart Dairy Farm Management System is proposed to support modern dairy farming through continuous digital monitoring, intelligent analysis, and centralized farm management. Dairy farms require reliable monitoring of livestock health, milk production, feeding schedules, and environmental conditions to improve productivity and animal welfare.

Traditional dairy farm management relies heavily on manual observation, making it difficult to identify early signs of disease, optimize feeding schedules, maintain accurate production records, and react quickly to changes in cattle behaviour. The proposed system provides a software foundation for integrating AI techniques, IoT and wearable sensor data, milk records, feeding recommendations, and farm analytics.

The current implementation is a prototype developed using FastAPI with REST API functionality. Git is used for version control, while Docker and Docker Compose are included for containerization and reproducible deployment. The disease-risk endpoint demonstrates a prototype decision model based on health thresholds; a trained AI/ML model and live IoT integration are identified as future enhancements.

1.2 Project Objectives

The architecture should remain extensible so that future AI models, IoT devices, databases, and mobile interfaces can be integrated without redesigning the complete application.

The system is intended to convert raw farm observations into understandable recommendations, alerts, and summary information for decision-making.

The objectives also support centralized record management so that farmers can compare health and production information over time.

The main objectives are:

1. Monitor cattle health continuously through recorded sensor readings.
2. Provide disease-risk prediction and early warning support.
3. Optimize and manage feeding schedules.
4. Track milk production records.
5. Generate farm performance and analytics summaries.
6. Improve animal welfare through early monitoring.
7. Reduce operational costs through data-driven management.
8. Enhance overall dairy productivity.
9. Provide a REST-based backend using FastAPI.

1.3 Functional Requirements

Future versions should validate input data, handle errors consistently, authenticate users, and store records in a persistent database.

Analytics services should provide summarized information that helps farmers identify trends in cattle health, milk production, and operational performance.

The disease-risk service should be treated as a decision-support feature and not as a replacement for veterinary diagnosis.

The API should accept structured cattle and farm data and return predictable JSON responses that can later be consumed by web or mobile dashboards.

The current prototype provides the following functionality:

REST API using FastAPI.

/health endpoint for application health monitoring.

/cattle endpoint for sample cattle records.

/disease/predict endpoint for prototype health-risk prediction.

/feeding/{cattle_id} endpoint for feeding recommendations.

/milk endpoint for recording milk production.

/analytics/summary endpoint for farm performance data.

Automatic Swagger API documentation through /docs.

Docker-based application containerization.

Docker Compose-based deployment configuration.

Git-based version control and feature development.

Future enhancements may include trained machine-learning disease models, real-time wearable and IoT sensor integration, environmental monitoring, database persistence, alerts, authentication, role-based access control, cloud deployment, and advanced reporting.

1.4 Expected Outcomes

Future deployment with real farm data should allow the outcomes to be evaluated using measurable indicators such as disease-alert accuracy, milk yield improvement, feed efficiency, and response time.

The system can provide a foundation for comparing animal-level observations with farm-level productivity indicators.

The prototype is expected to demonstrate how software services can organize dairy-farm information into measurable and actionable outputs.

The expected outcome is a version-controlled and container-ready backend prototype that demonstrates the core digital services required for smart dairy farm management. The prototype can be consistently developed, tested, and deployed while providing a foundation for future AI and IoT expansion.

1. Improved livestock health monitoring.
2. Increased milk production through better management.
3. Reduced disease outbreaks through earlier risk identification.
4. Better farm management and record keeping.
5. Lower operational costs.
6. Enhanced animal welfare.
7. Data-driven farming decisions.
8. Increased farmer profitability.

Project Repository Structure

```
smart-dairy-project/  
├── app/  
│   └── main.py  
├── screenshots/  
├── Dockerfile  
├── docker-compose.yml  
└── requirements.txt
```

Figure 1.1 – Smart Dairy Farm Management System Project Structure

2. PROJECT STRUCTURE DESIGN

A modular structure will be particularly important when AI models, database access, sensor integrations, and reporting services are added.

The project structure separates application logic from deployment and documentation files, making the repository easier to maintain.

2.1 Repository Structure

As the project expands, directories such as models, schemas, services, database, ml, tests, and routers can be introduced.

The screenshots directory stores execution evidence used in the technical report and helps document the prototype validation process.

The app directory contains the primary FastAPI implementation, while deployment files remain at the repository root for easy container management.

The project repository separates application source code, dependencies, container configuration, and generated validation evidence.

```
smart-dairy-project/  
├── app/  
│   └── main.py  
├── screenshots/  
├── Dockerfile  
├── docker-compose.yml  
└── requirements.txt
```

2.2 Purpose of Major Files

Documentation and validation evidence improve project traceability and make the implementation easier for another developer to reproduce.

The requirements file provides a reproducible list of Python dependencies and can be extended when machine-learning or database libraries are introduced.

Keeping deployment configuration separate from application logic makes it easier to update the runtime environment without unnecessarily changing business logic.

File/Folder	Purpose
app/	Contains application source code.
app/main.py	Defines the FastAPI application and Smart Dairy API endpoints.
requirements.txt	Defines Python dependencies.
Dockerfile	Defines instructions for building the Docker image.
docker-compose.yml	Defines the Docker Compose deployment configuration.
screenshots/	Contains validation screenshots generated from actual program execution.

The structure is suitable for the current prototype because it is simple and maintainable. As the system grows, additional modules such as routers, services, models, schemas, database layers, machine-learning pipelines, IoT adapters, and automated tests can be introduced.

3. GIT REPOSITORY INITIALIZATION

A remote repository also provides a backup location and supports collaboration when multiple team members work on separate modules.

Repository initialization establishes a controlled baseline from which all subsequent development can be tracked.

3.1 Repository Creation

After initialization, the remote repository can be used to synchronize commits and preserve the project's development history.

The initial repository should contain the minimum working version of the Smart Dairy application before feature branches are created.

Git is used as the version-control system for the Smart Dairy Farm Management System. The project can be initialized as a Git repository and connected to a remote GitHub repository for centralized synchronization and collaboration.

The initial project version can be recorded using the commit: Initial project setup.

3.2 Git Initialization Workflow

After a feature is tested, it can be merged into the development branch and verified again before becoming part of the stable release.

Before pushing changes, developers should check repository status and review the staged files to avoid committing unnecessary files.

Each feature should be developed through a separate branch and committed in small, meaningful stages.

```
git init
git status
git add .
git commit -m "Initial project setup"
git remote add origin <repository-url>
git push -u origin master
git checkout -b develop
git checkout -b feature/health-monitoring
```

The initial commit establishes the baseline version of the project. The develop branch can be used for integration, while feature branches can be used for isolated implementation of health monitoring, disease prediction, feeding management, milk tracking, and analytics.

3.3 Essential Git Files

Configuration files containing passwords, API keys, or private credentials should never be committed to the public repository.

The repository should also exclude virtual environments, cache folders, temporary files, local secrets, and generated artifacts that do not belong in source control.

The .git directory stores repository information including commit history, branches, and configuration.

A .gitignore file can prevent generated files such as Python cache files and local environment files from being tracked. A GitHub remote repository can provide centralized storage and synchronization of project changes.

4. GIT BRANCHING STRATEGY

This approach also makes it easier to review individual changes and identify the source of defects.

Feature-based branching makes it possible to develop health monitoring, disease prediction, feeding, milk tracking, and analytics independently.

4.1 Selected Branching Strategy

A separate feature branch reduces the possibility that experimental AI or IoT changes will interrupt the stable application.

The stable branch should contain only tested versions, while the development branch acts as the integration area for completed features.

A feature-based branching strategy is suitable for the project:

master



develop



feature/health-monitoring

feature/disease-prediction

feature/farm-analytics

The master branch represents the stable baseline, develop is used for integration, and feature branches support isolated implementation. This strategy reduces the risk of directly modifying the stable branch and supports independent development of the Smart Dairy modules.

4.2 Branch Creation and Integration

Merge conflicts should be resolved carefully and followed by another application test to confirm that integration has not introduced regressions.

Before merging, each feature should be tested through its relevant API endpoints and checked for compatibility with existing functionality.

The develop branch can be created from master. Individual feature branches can then be created for health monitoring, disease prediction, feeding recommendations, milk tracking, and analytics. After implementation and testing, the feature changes can be merged into develop.

5. VERSION CONTROL WORKFLOW

The workflow can later be connected to CI/CD so that automated tests run whenever changes are pushed.

A consistent workflow improves traceability because each change can be associated with a branch, commit, and integration step.

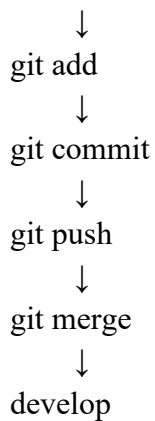
5.1 Git Workflow

The workflow encourages incremental development, making it easier to identify and revert a problematic change.

Developers should use descriptive commit messages that explain what changed rather than using vague messages such as update or changes.

The project follows the workflow:

Feature Development



This workflow allows changes to be developed separately, recorded as commits, synchronized with a remote repository, and integrated into the development branch.

5.2 Major Git Operations

Regular pushes also protect development progress by maintaining an up-to-date remote copy of the project.

The selected commands cover the essential lifecycle of feature development, from staging and committing changes to synchronizing and integrating them.

Git Operation	Purpose
git add	Stages project changes.
git commit	Records a version of the changes.
git push	Synchronizes local changes with GitHub.
git checkout / git switch	Switches or creates branches.
git merge	Integrates feature changes into another branch.

6. COMMIT HISTORY ANALYSIS

A well-maintained history can be used as evidence of incremental development in a technical assignment.

Commit history provides a chronological record of how the Smart Dairy prototype evolved from initial setup to API implementation and deployment configuration.

6.1 Commit History

Small feature-focused commits are easier to review than one very large commit containing unrelated changes.

The sequence of commits should reflect logical milestones such as API creation, health monitoring, disease prediction, analytics, containerization, and documentation.

A chronological commit history should represent the major stages of development. Suggested commits for this prototype include:

Initial project setup



Add Smart Dairy API endpoints



Add disease-risk prediction endpoint



Add Docker and Compose configuration



Add project documentation and validation evidence

6.2 Commit Message Quality

Messages should preferably use a consistent action-oriented style, for example Add disease-risk prediction endpoint or Add Docker Compose configuration.

Good commit messages improve maintainability because future developers can understand the reason for a change without examining every line of code.

Commit messages should clearly describe the purpose of the corresponding changes. Examples include Add Smart Dairy API endpoints, Add disease-risk prediction endpoint, Add Docker and Compose configuration, and Add project documentation. Clear commit messages improve traceability and make project history easier to understand and maintain.

7. DOCKER ENVIRONMENT DESIGN

The same container configuration can later be used as a basis for staging or cloud deployment.

Containerization is useful for ensuring that the FastAPI runtime and required Python packages are available consistently across different development machines.

7.1 Need for Docker

It also makes it easier to reproduce the same application environment during demonstration, testing, and deployment.

Docker reduces environment-related problems by packaging the application together with its runtime dependencies.

Docker is suitable for the project because the FastAPI application depends on a specific Python runtime and application dependencies. Without containerization, differences in Python versions or dependency configurations may cause inconsistent behaviour between development environments.

Docker provides an isolated and reproducible environment for executing the Smart Dairy Farm Management API.

7.2 Components Requiring Containerization

Possible future containers include a database service, AI inference service, IoT gateway, notification service, and monitoring component.

The current prototype contains one main application service, but the architecture can later be expanded into multiple services.

FastAPI Application

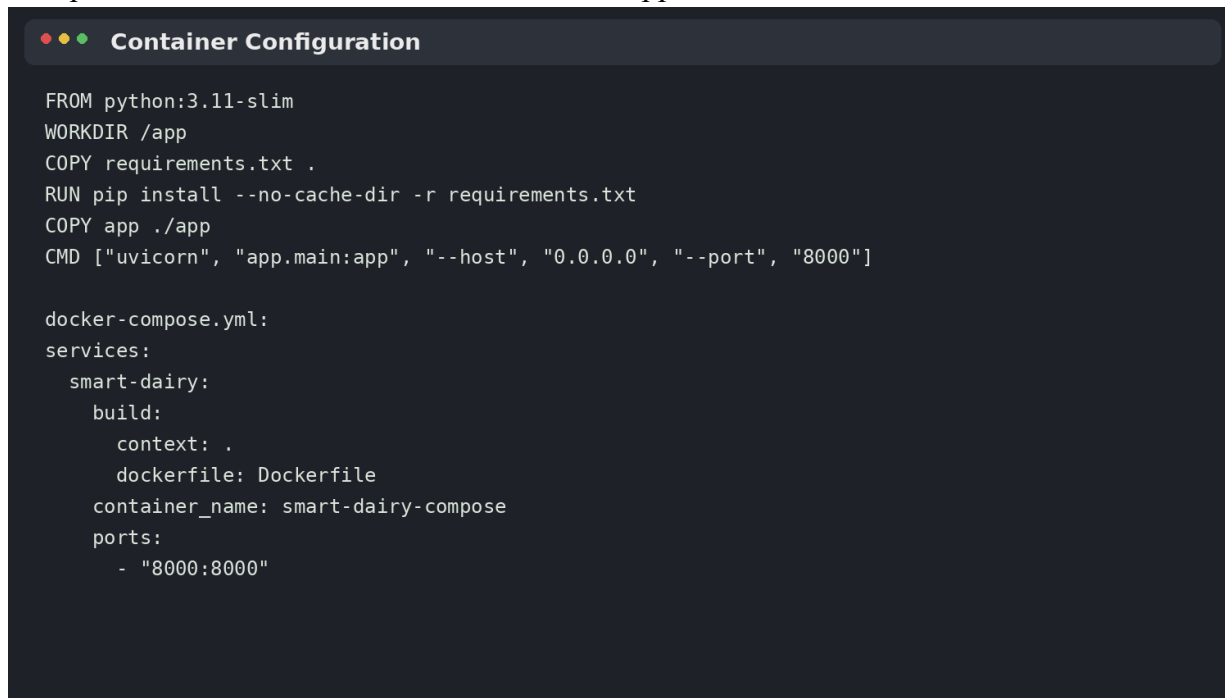


Python Runtime



Docker Container

The application source code and Python dependencies are packaged into the Docker image. Docker Compose can then be used to build and start the application service.



```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

docker-compose.yml:
services:
  smart-dairy:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: smart-dairy-compose
    ports:
      - "8000:8000"
```

Figure 7.1 – Dockerfile and Docker Compose Configuration

8. DOCKERFILE DEVELOPMENT

For production use, additional practices such as non-root execution, health checks, pinned dependency versions, and image-size optimization can be introduced.

The Dockerfile should remain minimal and deterministic so that application images can be rebuilt consistently.

8.1 Dockerfile

The application listens on port 8000 inside the container, matching the exposed service configuration.

The Dockerfile installs application dependencies before copying the source code, which can improve

Docker build-cache efficiency when source files change frequently.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 Dockerfile Instructions

The startup command launches Uvicorn and binds the service so that it can receive requests through the mapped host port.

Using a slim Python base image helps reduce unnecessary packages while retaining the Python runtime required by the FastAPI application.

Instruction	Purpose
FROM	Provides the Python 3.11 Slim base image.
WORKDIR	Sets /app as the container working directory.
COPY requirements.txt	Copies the dependency file into the container.
RUN	Installs the required Python packages.
COPY app	Copies the application source code.
CMD	Starts the FastAPI application using Uvicorn on port 8000.

The Dockerfile provides a reproducible procedure for building the application environment.

9. DOCKER COMPOSE DESIGN

The same Compose file can later be extended with additional services as the smart dairy architecture becomes more complex.

Docker Compose provides a simple way to describe how the application should be built and started, making demonstrations easier to reproduce.

9.1 Docker Compose Configuration

Port mapping allows the application to be accessed from the host machine while the service continues to run inside its isolated container environment.

The Compose configuration keeps the service definition in source control, allowing deployment settings to be reviewed alongside application code.

Docker Compose simplifies the building and execution of the application service. The current configuration defines the Smart Dairy application service and exposes port 8000.

```
services:
  smart-dairy:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: smart-dairy-compose
    ports:
      - "8000:8000"
```

9.2 Service and Network Configuration

Persistent volumes should be introduced when the application begins storing farm records in a database so that data survives container recreation.

A future multi-service version can use the automatically created Compose network for communication between the API, database, AI service, and IoT components.

The main service is smart-dairy. It builds the application using the Dockerfile and maps port 8000 of the host to port 8000 of the container. The application can therefore be accessed through `http://localhost:8000`.

For the current single-service prototype, no complex inter-service communication is required. Future versions may add database, message queue, machine-learning inference, and IoT gateway services.

10. APPLICATION DEPLOYMENT AND TESTING

Each important endpoint should be tested with both valid and invalid inputs before the system is considered ready for wider use.

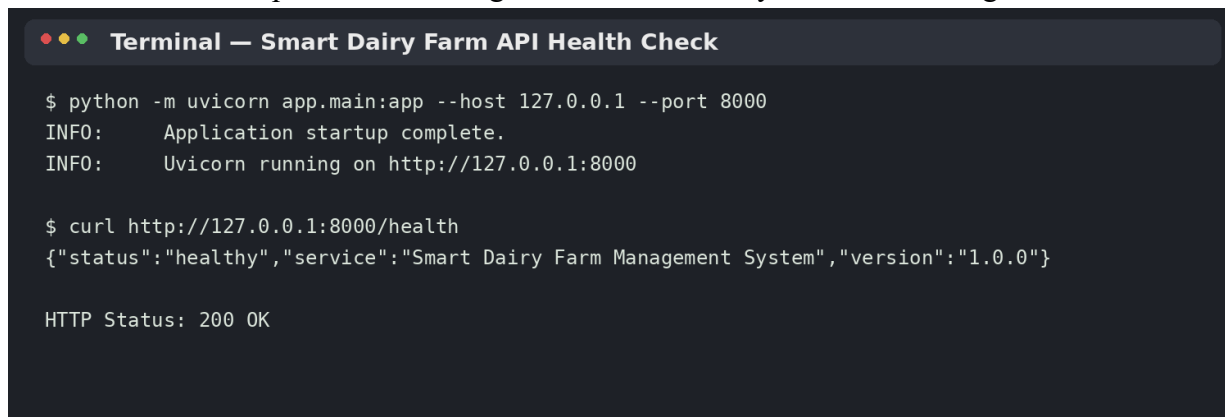
Testing confirms that the prototype is not only implemented in source code but also executable and accessible through its API interface.

10.1 Application Execution

A production version should also include deeper health checks for database availability, model availability, sensor connectivity, and other dependent services.

The health endpoint provides a simple first-level verification that the API process has started and can accept HTTP requests.

For validation, the FastAPI application was executed and tested locally. The health endpoint returned a successful HTTP response, confirming that the Smart Dairy API was running.



```
Terminal — Smart Dairy Farm API Health Check

$ python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000

$ curl http://127.0.0.1:8000/health
{"status":"healthy","service":"Smart Dairy Farm Management System","version":"1.0.0"}

HTTP Status: 200 OK
```

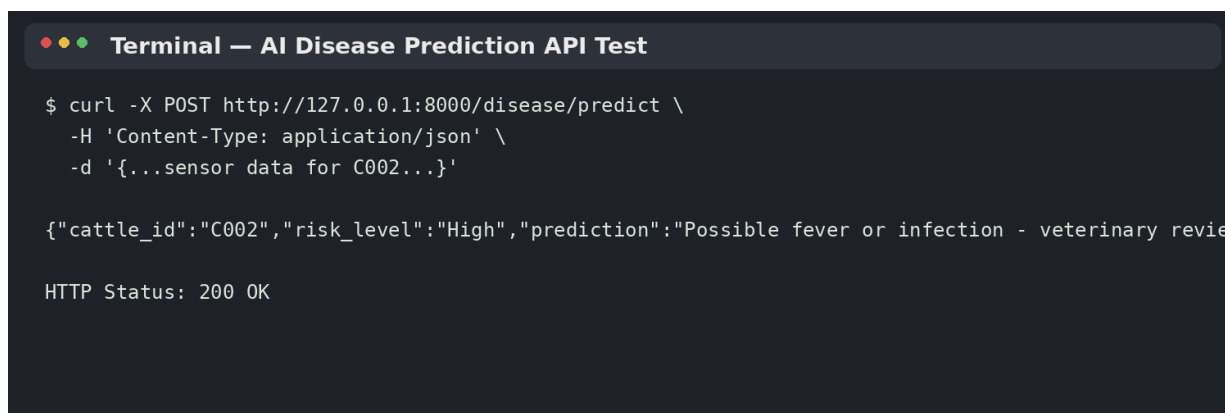
Figure 10.1 – Successful Smart Dairy API Health Check

10.2 Disease Prediction API Testing

A future AI model should be trained and evaluated using representative livestock datasets and should report appropriate performance measures before being used for operational decisions.

The prototype uses threshold-based logic to demonstrate the complete request-and-response flow for health-risk prediction.

A sample sensor reading for cattle C002 was submitted to the disease prediction endpoint. The prototype identified a high-risk condition because the submitted temperature and heart-rate values exceeded the configured thresholds. This validates the endpoint workflow; replacement with a trained ML model is a future enhancement.



```
Terminal — AI Disease Prediction API Test

$ curl -X POST http://127.0.0.1:8000/disease/predict \
-H 'Content-Type: application/json' \
-d '{...sensor data for C002...}'

{"cattle_id":"C002","risk_level":"High","prediction":"Possible fever or infection - veterinary review"}

HTTP Status: 200 OK
```

Figure 10.2 – Disease-Risk Prediction API Test

10.3 Farm Analytics Testing

Future dashboards can add daily, weekly, and monthly trends, comparison between cattle groups, milk-yield trends, and health-alert counts.

Farm analytics can help transform individual cattle records into management-level information.

The analytics endpoint was executed to obtain a summary of total cattle, healthy cattle, cattle requiring monitoring, average milk production, and estimated daily milk output.

A terminal window titled "Terminal — Farm Analytics Summary" with a dark background. It shows a command being executed: `$ curl http://127.0.0.1:8000/analytics/summary`. The output is a JSON object: `{"date": "2026-08-13", "total_cattle": 120, "healthy_cattle": 112, "monitoring_required": 8, "average_milk_li`. Below the JSON, it says "HTTP Status: 200 OK".

```
Terminal — Farm Analytics Summary

$ curl http://127.0.0.1:8000/analytics/summary

{"date": "2026-08-13", "total_cattle": 120, "healthy_cattle": 112, "monitoring_required": 8, "average_milk_li

HTTP Status: 200 OK
```

Figure 10.3 – Farm Analytics Summary API Test

10.4 Application Source Code

Keeping endpoint logic organized will make it easier to move business logic into dedicated service modules as the project grows.

The source-code evidence demonstrates the relationship between the API design and the functionality tested during execution.

The following screenshot shows the FastAPI application implementation used during the prototype execution and validation.

app/main.py — FastAPI Application

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from datetime import date
4
5 app = FastAPI(title='Intelligent AI-Based Smart Dairy Farm Management System', version='1.0.0')
6
7 class SensorReading(BaseModel):
8     cattle_id: str
9     temperature_c: float
10    heart_rate_bpm: int
11    rumination_minutes: int
12    activity_score: float
13
14 class MilkRecord(BaseModel):
15     cattle_id: str
16     litres: float
17     session: str
18
19 @app.get('/')
20 def root():
21     return {'message': 'Intelligent AI-Based Smart Dairy Farm Management System'}
22
23 @app.get('/health')
24 def health():
25     return {'status': 'healthy', 'service': 'Smart Dairy Farm Management System', 'version': '1.0.0'}
26
27 @app.get('/cattle')
28 def cattle():
29     return [
30         {'id': 'C001', 'name': 'Lakshmi', 'status': 'Normal', 'last_milk_litres': 18.5},
31         {'id': 'C002', 'name': 'Nandini', 'status': 'Monitor', 'last_milk_litres': 15.2},
32         {'id': 'C003', 'name': 'Ganga', 'status': 'Normal', 'last_milk_litres': 20.1},
33     ]
34
35 @app.post('/disease/predict')
36 def predict_disease(r: SensorReading):
37     risk = 'Low'
38     prediction = 'No immediate disease risk detected'
39     if r.temperature_c >= 39.5 or r.heart_rate_bpm >= 100:
40         risk = 'High'; prediction = 'Possible fever or infection - veterinary review recommended'
41     elif r.rumination_minutes < 300 or r.activity_score < 0.4:
42         risk = 'Medium'; prediction = 'Possible digestive or activity-related issue - monitor closely'
43     return {'cattle_id': r.cattle_id, 'risk_level': risk, 'prediction': prediction}
44
45 @app.get('/feeding/{cattle_id}')
```

Figure 10.4 – Smart Dairy Farm Management FastAPI Source Code

11. BENEFITS OF GIT AND DOCKER

Git manages the evolution of the source code, while Docker manages the reproducibility of the application runtime.

Using Git and Docker together creates a more controlled development lifecycle for the Smart Dairy application.

11.1 Benefits of Git

Version history also provides evidence of individual development stages and makes rollback possible when necessary.

Git supports experimentation because developers can create branches without changing the stable baseline.

Version Management: Records project changes through commits.

Branch-Based Development: Allows features to be developed independently.

Collaboration: GitHub can provide a centralized remote repository.

Change Tracking: Commit history records project evolution.

Recovery: Previous versions can be reviewed when required.

11.2 Benefits of Docker

Containerized deployment also provides a practical foundation for later migration to cloud or server infrastructure.

Docker reduces dependency conflicts and simplifies application setup on a new machine.

Portability: The application runs in a consistent environment.

Dependency Isolation: Dependencies are separated from the host environment.

Reproducibility: The Dockerfile can recreate the application environment.

Deployment Simplicity: Docker Compose simplifies application deployment.

Maintainability: Container configuration can be maintained as project code.

Together, Git and Docker improve version management, collaboration, portability, reproducibility, and deployment consistency for the Smart Dairy Farm Management System.

12. CHALLENGES AND FUTURE IMPROVEMENTS

System validation should involve realistic data, repeated testing, and appropriate domain review before operational decisions are automated.

The most important challenge is ensuring that technology-driven recommendations remain reliable enough for real farm operations.

12.1 Challenges

Data security becomes increasingly important when farm records and connected devices are accessible over networks.

Feeding optimization must consider multiple factors rather than relying on a single measurement.

Disease prediction requires high-quality labeled data and careful evaluation to reduce false alarms and missed cases.

Sensor readings may contain missing values, noise, or device errors, so preprocessing and validation will be necessary.

1. Collecting reliable livestock sensor data from wearable devices.
2. Distinguishing between prototype rule-based prediction and a validated trained AI model.
3. Maintaining real-time communication between sensors, backend services, and dashboards.
4. Designing feeding recommendations that account for age, weight, production stage, and health condition.
5. Securing farm data and controlling user access.
6. Containerizing multiple future services such as databases, AI inference, and IoT gateways.
7. Validating the system with real farm data before production deployment.

12.2 Future Improvements

Cloud deployment can enable remote access while requiring appropriate authentication, authorization, monitoring, and security controls.

Database integration can provide persistent historical records and support more advanced analytics.

A web or mobile dashboard can provide farmers with alerts, charts, cattle profiles, feeding recommendations, and production summaries.

IoT integration can continuously transmit temperature, heart rate, activity, rumination, and other supported measurements.

A trained AI/ML pipeline can replace the prototype threshold logic after sufficient labeled livestock data is collected.

1. Database integration for persistent cattle, sensor, feeding, and milk records.
2. Authentication and role-based authorization.
3. Real-time wearable and IoT sensor integration.
4. Trained machine-learning models for disease prediction.
5. Automated feeding optimization.
6. Environmental monitoring for temperature, humidity, and air quality.
7. Alert and notification services for abnormal health conditions.
8. Automated unit and integration testing.
9. CI/CD automation.
10. Cloud deployment and mobile dashboard integration.

CONCLUSION

The next development stage should focus on persistent data storage, validated AI models, real sensor integration, automated testing, and farmer-oriented dashboards.

The proposed system combines software engineering practices with a practical dairy-farm management use case and provides a clear path from prototype APIs to a larger intelligent farming platform.

The Intelligent AI-Based Smart Dairy Farm Management System prototype demonstrates the practical application of software engineering, version control, and containerization concepts to a modern dairy management problem.

The project provides API endpoints for health monitoring, cattle information, prototype disease-risk prediction, feeding recommendations, milk production recording, and farm analytics. Git can support repository management, branching, commits, feature development, and integration. Docker and Docker Compose provide a reproducible deployment foundation for the FastAPI application.

Successful API execution and HTTP 200 responses demonstrate that the current prototype can be started and validated. The project provides a foundation for future integration of real IoT wearable sensors, persistent databases, trained AI/ML disease models, automated feeding optimization, alerting, cloud deployment, and advanced farm dashboards.